Resumen teórico — 2do parcial Redes (RCCD 2C 2026)

Teóricas 12 (Congestión), 13 (Aplicación), 14 (Seguridad + Blockchain), 15 (Cómputo distribuido, Cloud/Fog/Edge), 15.1 (Lamport) y 15.2 (TrueTime). Pensado para la pregunta teórica: definiciones, por qués y trade-offs. ★ = lo más preguntable.

> **Cómo contestar una pregunta teórica:** (1) definir el concepto en una línea, (2) decir qué problema resuelve, (3) dar el trade-off o la limitación, (4) un ejemplo concreto. Casi todo en esta materia es "X resuelve Y, a cambio de Z".

1. Congestión (teórica 12)

- **Congestión** = sobrecarga **sostenida**: la demanda supera la capacidad de algún recurso (enlace, buffer) ⇒ se degrada la QoS. Síntomas: **pérdidas** y **delay creciente**.
- ★ **Control de flujo vs control de congestión**: flujo protege al **receptor** (punto a punto, RWND la anuncia el receptor); congestión protege a la **red** (global, CWND la estima el emisor). Se envía `min(CWND, RWND)`.
- **Knee** (codo): el throughput deja de crecer y el delay se dispara. **Cliff** (precipicio): colapso, goodput → 0. Potencia = throughput / delay, máxima cerca del knee. Evitar congestión = operar en el knee; controlarla = no caerse del cliff.
- **Por qué no alcanzan los arreglos obvios**: buffers infinitos ⇒ el delay crece hasta que vencen los timeouts ⇒ duplicados ⇒ más carga. Ruteo solo reparte (y si mira la carga, oscila). Más BW solo corre el problema.
- **Taxonomía**: lazo abierto (prealocar, circuitos) vs lazo cerrado: **implícito** (TCP infiere por pérdidas/delay) o **explícito** (ECN, DECbit, ATM ABR: la red avisa).
- **FIFO + Drop Tail**: el que llega con la cola llena se tira ⇒ todos los flujos pierden a la vez ⇒ **sincronización global** (todos bajan y suben juntos).
- ★ **RED** (Random Early Detection): promedio exponencial de la cola; por debajo de MinTh encola, entre MinTh y MaxTh descarta con probabilidad creciente (hasta MaxP ≈ 0,02), arriba de MaxTh descarta todo. Avisa temprano a flujos al azar ⇒ evita sincronización global y mantiene la cola corta.
- ★ **AIMD** (aumento aditivo, decremento multiplicativo): Chiu-Jain muestra que converge a **eficiencia y equidad**. Cortar a la mitad = vaciar la cola rápido + ser conservador.
- **Equidad de Jain**: $f = (\Sigma x)^2 / (n \cdot \Sigma x^2)$, entre 1/n (uno se lleva todo) y 1 (todos iguales).
- **M/M/1**: $\rho = \lambda/\mu < 1$; $E[T] = 1/(\mu - \lambda) \to \infty$ cuando $\rho \to 1$: el delay explota antes de llegar al 100 % de uso.
- **Mathis**: $BW \approx (MSS/RTT) \cdot \sqrt{3/2} / \sqrt{p}$ ⇒ TCP penaliza RTT largos y pérdidas (WiFi: pierde por errores de bit, no por congestión, y TCP lo malinterpreta).
- Slow start es "slow" comparado con TCP pre-1988, que mandaba toda la ventana de entrada.

2. Aplicación (teórica 13)

Base

- Protocolo de aplicación define: **tipos** de mensaje (petición/respuesta), **sintaxis** (campos), **semántica** (qué significan), **reglas** (cuándo y cómo se envían). Abiertos (RFC: HTTP, SMTP) o propietarios.
- Un proceso se identifica por **IP + puerto** (la IP sola no alcanza: muchos procesos por host). El socket es la "puerta" entre la app y el transporte del SO.
- **Cliente** habla primero; **servidor** espera en un puerto conocido.

★ ¿TCP o UDP? Requisitos de la aplicación

App	Pérdidas	Ancho de banda	Sensible al delay	Transporte
Archivos, mail, web	no tolera	flexible	no	TCP
Audio/video en tiempo real	tolera	mínimo (kbps–Mbps)	sí, ~100 ms	típicamente UDP (RTP, SIP)
Video almacenado	tolera	mínimo	pocos segundos	TCP (HTTP: DASH)
Juegos interactivos	tolera	pocos kbps	sí, ~100 ms	UDP o TCP

- TCP da: conexión, confiabilidad, control de flujo y de congestión. **No** da temporización ni BW mínimo. UDP no da nada de eso (pero no agrega handshake ni retransmisiones que lleguen tarde).
- **DASH** (MPEG-DASH): video en segmentos por HTTP, cada uno en varias calidades; el cliente elige el bitrate según el BW medido ⇒ streaming adaptativo sobre TCP.

DNS

- Base de datos **distribuida y jerárquica** (raíz → TLD → dominios → subdominios), escalable por delegación de **zonas** y caché. La app llama al resolvedor (`gethostbyname`), que manda UDP al servidor local.
- Registro = (nombre, TTL, clase, tipo, valor). SOA: primario, mail del admin, serial, timers.
- Consultas **recursivas** (resolvé vos todo) vs **iterativas** (decime a quién preguntar). Respuestas **autoritativas** (del dueño de la zona) vs **no autoritativas** (de caché).
- TLDs ~1500+, ccTLD ~300+; administra ICANN.

Mail

- Tres componentes: **agentes de usuario**, **servidores de correo** (buzón + cola de salida), **SMTP** entre servidores.
- SMTP: TCP puerto 25, transferencia **directa** servidor → servidor, tres fases (saludo, transferencia, cierre), comandos ASCII + respuestas con código. Mensajes en ASCII 7 bits.
- Comandos: `HELO · MAIL FROM · RCPT TO · DATA · (cuerpo terminado en ".") · QUIT`. Respuestas 250 OK, 354 empezá el cuerpo, 221 cierre.
- Formato RFC 822: cabecera (To, From, Subject) + línea en blanco + cuerpo ASCII. **MIME** agrega `MIME-Version`, `Content-Type` (tipo/subtipo) y `Content-Transfer-Encoding` (base64) para binarios.
- Leer el buzón: **POP3** (baja y suele borrar) vs **IMAP** (los mensajes quedan en el servidor, carpetas, sincroniza varios dispositivos).

HTTP

- Página = HTML base + objetos referenciados, cada uno con su URL (host + ruta). Cliente/servidor sobre TCP puerto 80.
- ★ **Sin estado**: el servidor no recuerda peticiones previas. Por qué: los protocolos con estado son complicados (si uno se cae, los estados quedan inconsistentes y hay que recomponerlos). El estado se agrega **afuera**: cookies, autorización en cada pedido.
- Métodos: GET, POST, HEAD (solo cabeceras, para validar). 1.1 agrega PUT (subir a esa URL) y DELETE.
- Códigos: 200 OK · 301 Moved Permanently (+ Location) · 400 Bad Request · 404 Not Found · 505 Version Not Supported. (También 201 Created, 304 Not Modified, 401 Unauthorized.)
- Tiempo de respuesta de un objeto = **2 RTT + transmisión** (1 RTT handshake TCP + 1 RTT pedido/primeros bytes).
- ★ **No persistente** (1.0): un objeto por conexión ⇒ 2 RTT por objeto + recursos del SO por conexión (los navegadores abren conexiones en paralelo). **Persistente** (1.1): la conexión queda abierta; **sin pipelining** 1 RTT por objeto; **con pipelining** 1 RTT para todos los referenciados.

- **Autorización**: como no hay estado, las credenciales van en cada pedido (`Authorization:`); sin ellas el servidor responde **401** + `WWW-Authenticate`.
- ★ **Cookies**, 4 componentes: (1) `Set-Cookie` en la respuesta, (2) `Cookie` en los pedidos siguientes, (3) archivo de cookies en el host del usuario (lo maneja el navegador), (4) base de datos en el sitio indexada por ese ID. Dan estado (carrito, sesión) sobre un protocolo sin estado; costo: privacidad/rastreo.

★ CDN

- Red de servidores que cachean el contenido de los proveedores **dentro o cerca de los ISPs**, para que el cliente lo baje localmente (clave para video: latencia y BW).
- El usuario se reparte por **redirección DNS**: el DNS de la CDN devuelve la IP del servidor más cercano (geográficamente o en latencia) según desde dónde pregunta el resolver.

3. Seguridad (teórica 14)

★ Propiedades (preguntar siempre: ¿cuáles provee esta tecnología?)

Propiedad	Qué garantiza
Confidencialidad	solo las partes especificadas entienden el mensaje
Integridad	no se modificó en tránsito
Originalidad	no es una copia repetida (anti-replay)
Temporalidad	no fue demorado maliciosamente
Autenticación	nadie asume la identidad de otro
Control de acceso (autorización)	solo ciertos usuarios hacen ciertas acciones
Disponibilidad	todo usuario tiene su oportunidad de ser atendido (contra DoS)
No repudio	nadie puede negar haber participado

Seguridad por capa

- Aplicación: SSH, PGP · Transporte: SSL/TLS · Red: **IPsec** · Enlace: WEP, WPA, WPA2.
- IPsec: **AH** (Authentication Header) da integridad, autenticación y no repudio (no cifra); **ESP** (Encapsulating Security Payload) da **confidencialidad**.

Criptografía clásica

- Plaintext → algoritmo parametrizado por una **clave** → ciphertext. Se asume un intruso pasivo que copia todo el ciphertext. Criptografía + criptoanálisis = criptología. **Cifrado** (carácter a carácter / bit a bit) ≠ **código** (reemplaza palabras: Navajo).
- **Sustitución** (enmascara, preserva el orden): César (rotación fija), monoalfabética ($26! \approx 4 \cdot 10^{26}$ claves, pero se rompe por **estadística de frecuencias** del idioma), polialfabética (un alfabeto por letra). **Transposición** (reordena sin enmascarar): clave sin letras repetidas (MEGABUCK) numera columnas; se escribe por filas y se lee por columnas.
- ★ **Kerckhoffs**: el algoritmo es público, solo la clave es secreta. La "seguridad por oscuridad" nunca funciona.
- **Factor de trabajo**: crece **exponencialmente** con el largo de la clave. 64 bits = juguete; 128/256 comercial; ≥ 256 crítico.
- **One-time pad**: clave aleatoria tan larga como el mensaje + XOR ⇒ **inviolable** (Shannon, teoría de la información). Problema: distribuir y proteger la clave ⇒ impráctico. Criptografía cuántica (BB84) apunta a distribuir claves así.

Simétrica

- `C = E_K(P)`, `P = D_K(C)` ⇒ `P = D_K(E_K(P))`. Misma clave en los dos extremos.

- **Cifrado de bloque**: bloque de n bits → n bits, con sustitución + permutación iteradas. **DES** (IBM 1977): bloques de 64 bits, 16 rondas con subclaves K1..K16 derivadas de una clave; quedó corto de clave (fuerza bruta). **3DES**: `C = E_K3(D_K2(E_K1(P)))`. **AES** = el mínimo recomendado hoy.
- ★ Problema: ¿cómo acordar la clave con alguien que nunca viste? ⇒ asimétrica o un KDC.

Asimétrica (clave pública)

- Diffie-Hellman 1976, RSA 1978. Pública (cifrar, la conoce todo el mundo) y privada (descifrar, solo el dueño). Requisito: inviable derivar la privada de la pública o de pares plaintext/ciphertext.
- ★ **RSA**: (1) primos grandes p, q; (2) $n = p \cdot q$, $z = (p-1)(q-1)$; (3) e < n coprimo con z; (4) d con $e \cdot d \bmod z = 1$; (5) pública (n, e), privada (n, d). Cifrar $c = m^e \bmod n$, descifrar $m = c^d \bmod n$. Seguridad: factorizar n es costoso. Ejemplo de clase: p=3, q=11 ⇒ n=33, z=20, e=3, d=7.
- ★ Propiedad clave: $K^-(K^+(m)) = m = K^+(K^-(m))$. Pública primero = confidencialidad; **privada primero = firma**.
- Asimétrica es **cara** ⇒ en la práctica, asimétrica para autenticar y acordar una clave de sesión simétrica (TLS).

★ Firma digital y hash

- Firmar = cifrar con la **privada**: solo el dueño pudo producir $K^-_B(m)$ ⇒ autenticidad, integridad y **no repudio**.
- Cifrar el mensaje entero con RSA es caro ⇒ se firma el **digest**: $K^-_B(H(m))$. Verificar: recalcular H(m) y comparar con $K^+_B(firma)$.
- Hash: muchos a uno, salida de tamaño fijo, rápido. Propiedades: **preimagen** (dado y, inviable hallar x con H(x)=y), **segunda preimagen** (dado x, inviable hallar otro x′ con el mismo hash), **colisiones** (inviable hallar cualquier par). Efecto avalancha.
- MD5 (128 bits) y SHA-1 (160 bits): con colisiones encontradas. SHA-2 (256/512) vigente.

★ Autenticación con clave compartida (challenge-response)

1. A → B: "soy A". 2. B → A: R_B (nonce aleatorio, en claro). 3. A → B: $K_{AB}(R_B)$ ⇒ B sabe que es A. 4. A → B: R_A. 5. B → A: $K_{AB}(R_A)$ ⇒ A sabe que es B. Opcional: A manda una clave de sesión cifrada con K_AB.

- El nonce aleatorio y largo (128 bits) hace improbable reusar una respuesta vieja (**anti-replay**).
- Versión simplificada (3 mensajes: identidad + challenge juntos) ⇒ vulnerable al **ataque por sesiones (reflexión)**: T se hace pasar por A, recibe R_B, abre una **segunda sesión** mandándole a B su propio R_B como challenge, B lo cifra, y T usa esa respuesta en la primera sesión.
- **Reglas de diseño**: el que inicia prueba su identidad primero · claves distintas por dirección (K_AB y K′_AB) · challenges de conjuntos distintos (pares/impares) · resistir sesiones paralelas.
- **KDC** (Key Distribution Center): cada uno comparte una clave solo con el KDC. A → KDC: $A, K_A(B, K_S)$; KDC → B: $K_B(A, K_S)$. Resuelve las N^2 claves, pero el KDC es punto central de confianza y falla.

4. Blockchain (teórica 14, 2da parte)

> **Idea central:** registro **append-only**, **replicado** entre nodos que **no confían entre sí**, que acuerdan un **único orden** sin autoridad central. Hash → integridad · firmas → autoría · **consenso** → **orden** (lo nuevo).

- ★ **Qué no da la firma**: orden (cuál fue primero), unicidad (que no haya otro mensaje firmado que lo contradiga), acuerdo (que todos vean el mismo historial). ⇒ **Doble gasto**: Alice firma tx_1 (moneda #42 → Bob) y tx_2 (#42 → Carol); las dos firmas verifican, la criptografía no decide cuál vale. Solución clásica: un tercero (banco).
- **Encadenar por hash**: cada bloque lleva $prev = H(bloque\ anterior)$. Alterar una tx del bloque 41 cambia su hash ⇒ el prev del 42 no coincide ⇒ toda la cadena posterior queda inválida.
- **Árbol de Merkle**: la raíz (32 bytes) va en el header; cambiar cualquier tx cambia la raíz. Probar que una tx está incluida: **$\log_2(n)$ hashes** ⇒ clientes livianos (SPV) sin bajar el bloque.

- **Transacciones**: dirección = hash(clave pública), sin registro de usuarios. Gastar = referenciar salidas previas no gastadas (UTXO) y firmar con la privada (ECDSA/Schnorr). Cada nodo verifica la firma y que la salida exista y no esté gastada. "La firma responde quién; el consenso, cuál primero."
- ★ **Consenso en red abierta**: generales bizantinos (Lamport-Shostak-Pease 1982) exigen **n ≥ 3f + 1** con identidades **conocidas**. En una red abierta, crear identidades es gratis ⇒ **ataque Sybil** ("un nodo, un voto" no sirve). Nakamoto 2008: votar con un recurso **escaso y verificable** (cómputo).
- ★ **Proof of Work**: buscar nonce con `SHA256(SHA256(header)) < target`. Caro de producir, **barato de verificar** (un hash). Dificultad ajustada cada 2016 bloques para ~10 min/bloque. Gana la cadena con **más trabajo acumulado**; minar es una lotería proporcional al cómputo.
- **Confirmaciones**: si el atacante tiene q < 50 % del cómputo, su chance de alcanzar a los honestos cae **exponencialmente** con z bloques de ventaja (paseo aleatorio). Con q ≥ 50 % gana siempre (**ataque del 51 %**). "6 confirmaciones (~1 h)" es convención, no teorema.
- **Mirada de redes**: difusión por **gossip** (nadie ve la red completa); **forks transitorios** cuando dos mineros encuentran bloque casi a la vez (el de la rama perdedora queda huérfano). Bloques más rápidos ⇒ más forks y trabajo perdido (Bitcoin ~7 tx/s).

	Proof of Work	**Proof of Stake**	**BFT (PBFT)**
Recurso escaso	cómputo (energía)	capital en garantía	identidad conocida
Red	abierta	abierta	permisionada
Finalidad	probabilística	económica (checkpoints)	inmediata
Tolera	< 50 % del cómputo	< 1/3 del stake	f < n/3 nodos
Ejemplo	Bitcoin	Ethereum (desde 2022)	CometBFT (Cosmos)

- PoS: el validador que hace trampa pierde su garantía (**slashing**).
- **Ethereum** = máquina de estados replicada (`estado_{n+1} = δ(estado_n, txs)`) entre nodos que no se conocen. Smart contracts (código + estado en la cadena, todos lo ejecutan en la EVM); **gas** acota el cómputo y corta loops infinitos; bugs inmutables (The DAO 2016).
- ★ **¿Cuándo conviene?** Muchos escriben y no confían entre sí + no hay un tercero aceptable para todos + historial verificable públicamente. Si hay un tercero confiable, una **base de datos replicada** es más simple, rápida y barata.

5. Cómputo distribuido: de la nube al edge (teórica 15)

> **Tesis de la clase:** cloud, edge y fog no son productos sino respuestas distintas a las mismas preguntas: dónde está el estado, cómo se replica, qué se sacrifica ante una partición y cuánto cuesta coordinar. Las restricciones no cambian; cambia **dónde se ubica el cómputo** respecto de datos y usuarios.

Qué es un sistema distribuido

- Lamport: "aquel en el que la falla de una computadora que ni sabías que existía deja inutilizable la tuya". Tanenbaum: computadoras independientes que se presentan como un único sistema coherente.
- ★ **Tres rasgos** (Coulouris): **concurrencia** (carreras, hay que coordinar) · **sin reloj global** (cada reloj deriva, no hay "ahora" único) · **fallas independientes** (algo falla y el resto sigue; a veces nadie sabe que falló). Comunicación **solo por mensajes**: lo que un nodo sabe del otro puede estar desactualizado.
- **Por qué distribuir**: escalar, tolerar fallas (redundancia), localidad (cerca de usuarios/datos), costo (muchas máquinas comunes). Escalar vertical tiene techo; horizontal no, pero exige coordinar.
- ★ **Amdahl**: `S(n) = 1 / ((1 − p) + p/n)`, con n → ∞ `S → 1/(1 − p)`. Con p = 0,95: S(8) = 5,9 · S(64) = 15,4 · S(1024) = 19,6 ⇒ un 5 % secuencial (coordinación, lock, consenso) limita a **20×**. Gustafson: si el problema crece con n, mejora.

★ Modelos de sistema (todo algoritmo es correcto *respecto de un modelo*)

Tiempo	Fallas de procesos	Canales
Síncrono: cotas conocidas **Parcialmente síncrono**: cotas existen pero no se conocen, o valen desde algún momento **Asíncrono**: sin cotas	**Crash**: se detiene y no vuelve **Crash-recovery**: vuelve con estado persistente **Bizantina**: arbitraria, incluso maliciosa	**Confiables**: todo llega una vez **Pérdidas equitativas**: si reintentás, llega **Arbitrarios**: pérdida, duplicación, reorden

Las 8 falacias (Deutsch, Gosling)

La red es confiable · la latencia es cero · el ancho de banda es infinito · la red es segura · la topología no cambia · hay un solo administrador · el costo de transporte es cero · la red es homogénea. Las tres primeras son las que más cuestan (**una llamada remota no es una llamada local**); en el edge las ocho son todavía más falsas.

★ Resultados de imposibilidad

Resultado	Dice
Dos Generales (1975/78)	con un canal que puede perder mensajes, ningún protocolo garantiza que dos procesos acuerden actuar juntos: siempre falta "el último ACK"
FLP (1985)	en un sistema **asíncrono**, ningún algoritmo **determinista** de consenso termina siempre si puede fallar **un solo** proceso por crash
CAP (Brewer 2000, prueba 2002)	ante una **partición**, un sistema replicado elige entre **consistencia** (linealizabilidad) y **disponibilidad**
PACELC (Abadi 2012)	si hay Partición: A o C; si no (Else): **Latencia o Consistencia**. La partición es rara; la latencia, permanente

Cómo se vive con esto: aleatoriedad, timeouts (sincronía parcial), detectores de falla, y elegir explícitamente qué sacrificar.

★ Replicar estado

- **Máquina de estados replicada** (Lamport 1978, Schneider 1990): mismos comandos en el mismo orden ⇒ mismo estado. El problema se reduce a **acordar el orden del log** = consenso (líder ordena, seguidores aplican).
- Réplicas para tolerar f fallas: **crash n ≥ 2f + 1** (Paxos, Raft) · **bizantina n ≥ 3f + 1** (PBFT). Por qué: dos mayorías cualesquiera se intersectan ⇒ siempre hay un testigo de la última decisión.
- **Quórums**: N réplicas, escribir en W, leer de R: **R + W > N** ⇒ toda lectura ve la última escritura. N=3, W=2, R=2 tolera una caída. Bajar R o W compra latencia a cambio de consistencia (PACELC).

★ Espectro de consistencia (de fuerte a débil)

Modelo	Qué garantiza	Costo	Ejemplo
Linealizable	cada operación parece instantánea en un punto entre su inicio y su fin	consenso en cada escritura	Spanner, etcd
Secuencial	un orden total común, compatible con el de cada proceso	orden total	memoria multiprocesador idealizada
Causal	si a pudo influir en b, todos ven a antes que b	metadatos causales	COPS, chats, redes sociales
Eventual	si cesan las escrituras, las réplicas convergen	ninguna coordinación síncrona	DNS, Dynamo, CRDTs

Hacia abajo: menos coordinación, menos latencia, más disponibilidad ante particiones, y más trabajo para el programador (conflictos, reconciliación).

El péndulo centralizar ↔ distribuir

Mainframe (C, 60–70) → cliente-servidor (D, 80s) → clusters/grid (D, 90s) → nube pública S3/EC2 (C, 2006) → edge/fog (D, 2009–14) → continuo dispositivo–edge–nube (2020s). ★ Lo mueve la relación entre **costo de computar, costo de comunicar y latencia tolerable**: red barata y rápida ⇒ centralizar; datos o latencia pesan más ⇒ distribuir.

La nube como sistema distribuido

- **Las fallas son la norma**: 10 000 servidores con MTBF de 30 años ⇒ 10 000 / (30 × 365) ≈ **0,9 fallas por día**. Replicar, reintentar y detectar no es opcional.
- **La latencia de cola domina**: fan-out a 100 servidores, cada uno lento el 1 % ⇒ $1 - 0{,}99^{100} \approx 63\ \%$ de los pedidos son lentos. El p99 de cada nodo es la mediana del sistema.
- **Llevar el cómputo a los datos**: MapReduce (el sistema reparte, reintenta y lanza copias de las tareas lentas). "El datacenter es la computadora."
- Piezas: configuración (etcd, Chubby) = consenso y elección de líder; bases globales (Spanner) vs clave-valor (Dynamo) = linealizabilidad vs disponibilidad.

★ Edge y fog

- La velocidad de la luz limita: en fibra ≈ 200 000 km/s ⇒ **1 ms de RTT ≈ 100 km**. Desde Buenos Aires: São Paulo ≥ 17 ms, Virginia ≥ 84 ms.
- **Cuatro motivos** para acercar el cómputo: **latencia** (control, AR, vehículos necesitan 1–20 ms) · **volumen** (1000 cámaras × 4 Mb/s = 4 Gb/s que no conviene subir) · **autonomía** (seguir operando si se corta la nube) · **soberanía** (datos que por ley o privacidad no salen).

Nivel	Cantidad	Latencia
Nube	decenas de regiones	50–150 ms
Fog / edge regional	cientos a miles de sitios	5–20 ms
Edge de acceso	decenas de miles	1–5 ms
Dispositivos	miles de millones	< 1 ms

- **Fog** = la jerarquía completa y la cooperación entre niveles (Cisco 2012). **Edge** = el borde de la red de acceso. **Cloudlet** = "datacenter en una caja" cerca del usuario (2009).
- Problemas nuevos del continuo: **placement** (asignar tareas a nodos con capacidad acotada minimizando latencia: NP-difícil ⇒ heurísticas) · **migración con movilidad** (el estado sigue al usuario; migrar en vivo converge si la tasa de escritura < la de copia) · **consistencia entre niveles** (fuerte = latencia de nube; eventual = conflictos ⇒ el edge opera abajo en el espectro) · **autonomía ante particiones** (opera aislado y reconcilia; **CRDTs** convergen sin coordinar, relojes híbridos ordenan sin sincronía perfecta).

6. Relojes de Lamport y vectoriales (15.1)

- ★ **"Ocurrió antes" (a → b)**: a y b en el mismo proceso con a primero; o a es el envío y b la recepción del mismo mensaje; o por transitividad. Si ni a → b ni b → a: **concurrentes** (ninguno pudo influir en el otro).
- ★ **Tres reglas** (un contador C por proceso): (1) antes de cada evento local o envío, `C := C + 1`; (2) el mensaje lleva `t = C`; (3) al recibir con marca t, `C := max(C, t) + 1`. El max asegura que la recepción quede después del envío aunque el receptor viniera atrasado.
- ★ Garantía: **a → b ⇒ C(a) < C(b)**. La recíproca **no** vale. Orden total: desempatar por id de proceso, (C, p).

```
p1:  a(1) ──m1──▶ p2        b(2) ──m2──▶ p3
p2:  c(1)   d(2)   e = max(2, 1) + 1 = 3     (gana el reloj local)
p3:                f = max(0, 2) + 1 = 3     (gana la marca del mensaje)
```

Evento	a	b	c	d	e	f
Lamport	1	2	1	2	3	3
Vectorial (p1,p2,p3)	(1,0,0)	(2,0,0)	(0,1,0)	(0,2,0)	(1,3,0)	(2,0,1)

- C(c) = 1 < C(b) = 2 pero **c y b son concurrentes**: Lamport no lo detecta.
- **Vectoriales** (Fidge, Mattern 1988): cada proceso cuenta los eventos de todos; al recibir, **máximo componente a componente** y +1 en la propia. **a → b ⇔ V(a) < V(b)** (todas ≤ y alguna <). c = (0,1,0) y b = (2,0,0) no son comparables ⇒ concurrentes. Costo: n enteros por mensaje.
- **Relojes físicos derivan**: cristal de 20 ppm ⇒ $20 \cdot 10^{-6} \times 86\,400$ s ≈ **1,7 s por día**. NTP: milisegundos en Internet; PTP: microsegundos en LAN. Nunca cero.

7. TrueTime de Spanner (15.2)

> **Idea:** ningún reloj sabe la hora exacta. TrueTime no lo oculta: devuelve un **intervalo** que garantiza contener la hora real, y cada commit **espera** hasta que la incertidumbre haya pasado. Convierte **incertidumbre en latencia**.

- API: `TT.now()` → [earliest, latest], ancho 2ε · `TT.after(t)`: t ya pasó seguro (t < now().earliest) · `TT.before(t)`: t no llegó seguro (t > now().latest).
- Spanner asume deriva de **200 µs/s** (10× peor que un cristal, por seguridad) y sincroniza cada **30 s** con time masters (GPS + relojes atómicos).
- ★ `ε = 1 ms + 0,2 ms/s × Δt` (Δt desde la última sincronización). Ejemplo: Δt = 15 s ⇒ ε = 4 ms, intervalo de 8 ms. ε entre ~1 y 7 ms.
- ★ **Commit wait**: elegir `s = TT.now().latest`; no liberar locks ni responder hasta que `TT.after(s)`. Cuesta ≈ **2ε** por commit (en parte se superpone con la ronda de Paxos).
- ★ **Consistencia externa**: si T1 hace commit en el nodo A y recién después (en tiempo real) arranca T2 en el nodo B, entonces $\mathbf{s_1 < s_2}$, para cualquier error de reloj dentro de ±ε. Peor caso (A adelantado ε, B atrasado ε): $s_1 \le r_1 + 2\varepsilon$, A confirma recién en $r_1 + 2\varepsilon$, T2 arranca después ⇒ $s_2 \ge r_2 > s_1$.
- ★ **vs Lamport**: Lamport ordena compatible con la causalidad que **viaja en mensajes**. TrueTime ordena también eventos **sin mensajes entre sí** (T1 y T2 relacionadas solo por un cliente externo) porque ancla los timestamps a la hora real.

8. Preguntas tipo teórica (autoevaluación)

1. ¿Por qué una firma digital no alcanza para evitar el doble gasto? ¿Qué agrega blockchain?
2. ¿Por qué PoW resiste un ataque Sybil y "un nodo, un voto" no? ¿Qué pasa con 51 %?
3. ¿Cuándo **no** conviene una blockchain?
4. ¿Qué propiedades da AH vs ESP? ¿Una firma da confidencialidad?
5. Kerckhoffs y por qué la longitud de la clave importa. ¿Por qué el one-time pad no se usa?
6. Explicar el ataque por reflexión y dos reglas de diseño que lo evitan.
7. Si a → b, ¿qué se puede decir de C(a) y C(b)? ¿Y si C(a) < C(b)? ¿Cómo lo resuelven los vectoriales?
8. ¿Qué es el commit wait, cuánto cuesta y qué garantiza que Lamport no?
9. CAP vs PACELC: ¿por qué PACELC dice que la latencia importa más?
10. ¿Cuántas réplicas para tolerar f crashes? ¿Y f bizantinos? ¿Por qué R + W > N?
11. Dar 3 motivos para hacer cómputo en el edge en vez de la nube, con un ejemplo cada uno.
12. Amdahl con p = 0,9: ¿cuál es el speedup máximo? (10×)
13. ¿Por qué HTTP es sin estado y cómo se agrega estado? ¿Cómo reparte usuarios una CDN?
14. Control de flujo vs control de congestión. ¿Por qué RED evita la sincronización global?